

BBR Under Mixed Workloads: Characterizing Short-Flow Latency and a Targeted Pacing Gain Improvement

Adri Katyayan (SBU ID: 117353314) | Mehataab Naazneen Mohammed (SBU ID: 117376528)
CSE 534 – Fundamentals of Computer Networks

Abstract

BBR is Google’s TCP congestion-control algorithm, and it is widely deployed because it can sustain high throughput without relying on packet loss as its main congestion signal. In steady state, however, BBRv1 periodically probes for extra bandwidth by pacing above the estimated bottleneck rate and then attempting to drain what it created. In practice, that drain is not perfect. A standing queue can persist at the bottleneck between probe cycles.

We ran experiments to measure how much that queue hurts short flows. In networking terms, short flows are “mice” and long bulk transfers are “elephants.” Our mixed-workload setup runs one BBR elephant for two minutes while ten sequential 1 MB mice transfers share the same bottleneck. We sweep nine combinations of buffer size and round-trip time. At the shallow 10 KB buffer, mice were effectively trapped behind the elephant’s queue: flow completion time rose from near-ideal values to double-digit seconds, with several configurations landing in the 12 to 16 second range.

Our proposed intervention was simple: recompile BBR’s kernel module with a smaller ProbeBW probe-up pacing gain. The stock value is 1.25 $\times$. We tested 1.20 $\times$, 1.15 $\times$, and 1.10 $\times$ as separate loadable Linux kernel modules built from the BBRv1 source. In the canonical dataset used in this report, all three modified gains improved shallow-buffer mice latency, and the strongest average FCT reduction at 10 KB came from the moderate 1.20 $\times$ variant. Because several Phase 3 runs preserved valid FCT data but lost elephant-goodput fields, our strongest conclusion is about latency rather than a final deployment recommendation. The overall result is still clear: BBR’s probe-up aggressiveness is a meaningful tuning knob for mixed-workload environments where short-flow latency matters.

1 Introduction

TCP congestion control is one of those problems networking researchers thought they had wrapped up by the 1990s. CUBIC, Reno, and the rest of that family kept the internet running for decades with a simple idea: if packets drop, back off. Then Google shipped BBR [1] and the whole framing shifted.

BBR does not wait for drops. Unlike loss-based algorithms, BBR actively measures two physical path properties: the bottleneck bandwidth (BtlBw) and the minimum round-trip propagation delay (RTprop). Multiply them together and you get the Bandwidth-Delay Product, which is exactly how much data should be in flight at any moment to saturate the link without filling any buffer. YouTube, Google Search, and Google’s internal WAN backbone all run it. On high-latency paths, the throughput gains over CUBIC are hard to argue with.

There is a structural problem in how BBR behaves during normal operation though. In its steady-state mode, called ProbeBW, BBR periodically tests whether more bandwidth has freed up by pushing 25% above its current estimate for one round-trip, then stepping back to 75% to drain whatever queue that created. The drain is supposed to clear things out before the next probe. It does not keep up. A small queue sits at the

bottleneck permanently between cycles. Scherrer et al. [4] proved analytically that this queue settles at around 1 BDP under deep buffers.

Any short flow arriving mid-probe waits behind it. A REST API call, a DNS lookup, a browser fetch. At a shallow bottleneck buffer, a 1 MB file that should finish in under 100 ms can take 12 to 16 seconds. That is not a rounding error; it is a three-order-of-magnitude degradation.

This is not a constructed edge case either. It is just what happens on any server running a background bulk transfer alongside normal short requests, which is most servers. The original BBR paper [1] measured bulk throughput only. The most thorough empirical follow-up [2] studied BBR alone or competing with CUBIC, not what it does to short flows sharing a pipe with it. We look at that directly, with two contributions:

- We characterize how much a BBR elephant degrades short-flow completion time across nine configurations: buffer sizes of 10 KB, 200 KB, and 10 MB, at RTTs of 10, 40, and 100 ms. The damage is worst at shallow buffers and does scale with RTT.
- We build three modified BBR kernel modules that reduce the probe-up gain from 1.25× to 1.20×, 1.15×, and 1.10×, re-run the full experiment sweep for each, and produce a tradeoff curve between short-flow latency and elephant throughput.

Section 2 covers BBR’s mechanics and the most relevant prior work. Section 3 states the problem and hypotheses. Section 4 presents the methodology and experimental setup. Section 5 presents the results and discussion. Section 6 concludes.

2 Background and Related Work

2.1 BBR Congestion Control

BBR was introduced by Google in 2016 and takes a fundamentally different approach to congestion control than loss-based algorithms such as CUBIC [1]. Instead of using packet loss as the main signal to slow down, BBR measures the path directly.

It tracks two quantities. $BtlBw$ is the maximum delivery rate observed over a sliding window, which serves as BBR’s estimate of bottleneck bandwidth. $RTprop$ is the minimum RTT observed over a longer window, which represents the base delay when no queue is present. Their product is the bandwidth-delay product, or BDP, which is the amount of data that should be in flight to fill the path without depending on persistent buffering.

The algorithm moves through four states:

- **Startup:** ramps up aggressively to find the bottleneck bandwidth.
- **Drain:** bleeds off the queue created during Startup.
- **ProbeBW:** the steady-state mode where BBR spends most of its time.
- **ProbeRTT:** periodically refreshes the $RTprop$ estimate.

ProbeBW is the state that matters most for this project. Roughly every eight RTTs, BBR cycles through a sequence of pacing gains. One phase sends at about 1.25× the estimated bottleneck rate to test whether more capacity has opened up. The next drops to 0.75× to drain the queue that was just created, followed by several phases at 1.0× to cruise. In principle, this should let BBR probe aggressively without leaving much queue behind. In practice, the drain phase often does not fully clear the backlog. Scherrer et al. showed through fluid models that the remaining queue stabilizes at roughly one BDP in steady state [4].

2.2 Related Work

Cao et al. provide the most relevant empirical baseline for this project [2]. Their Mininet study evaluates BBR across a wide range of buffer sizes, RTTs, and link rates. Our Phase 1 validation follows their shallow-buffer throughput trend as a sanity check. What their study does not directly examine is the mixed-workload case: what happens when a BBR elephant shares a bottleneck with short mice flows. That is the gap this project addresses.

Ware et al. provide the analytical foundation for understanding why shallow buffers are so damaging in this setting [3]. Their model explains how BBR’s probe-up behavior can dominate a bottleneck, causing competing flows to back off while BBR continues to push forward. The same logic helps explain our short-flow penalty: mice that arrive during or just after probe-up are forced to wait behind the elephant’s backlog.

Scherrer et al. refine the queueing picture by showing that BBRv1 can sustain a standing queue in steady state [4]. This matters for our RTT interpretation, because a larger RT_{prop} implies a larger BDP, and therefore a larger queue when the bottleneck can hold it.

On the gain-modification side, BBR-GC and related proposals show that BBR’s gain schedule is a legitimate intervention target [5]. Those systems focus mainly on fairness among long-lived flows. Our goal is different: we are not trying to make elephants fairer to one another, but to reduce the latency penalty they impose on mice.

We had also planned a BBRv3 comparison. That comparison could not be completed because the WSL2 kernel used in this project did not expose a usable BBRv3 module. We therefore treat BBRv3 as a platform limitation rather than a result axis in the final report.

3 Problem Statement

The stock BBRv1 ProbeBW cycle includes a probe-up phase that sends at roughly $1.25\times$ the estimated bottleneck bandwidth. This intentional overshoot helps BBR discover spare capacity, but it also risks building queues at the bottleneck. In a workload containing one sustained elephant flow and many short mice flows, even a modest standing queue can translate into large relative delays for mice traffic.

The central question in this project is therefore: Can a small reduction in BBRv1’s probe-up pacing gain reduce mice-flow completion time under mixed workloads while preserving most of the elephant’s throughput?

4 Approach

We decompose the project into three phases so that each later stage builds on a validated earlier stage.

4.1 Phase 1: Validation

Phase 1 reproduces the qualitative shallow-buffer trend from Cao et al. using a simpler setup than the original paper. The goal is not a claim of full paper-faithful reproduction, but to verify that our Mininet topology and traffic shaping produce buffer-sensitive behavior.

4.2 Phase 2: Mixed-Workload Characterization

Phase 2 introduces the main workload of interest: one long-running BBR elephant flow plus ten sequential mice flows. This phase measures how mice FCT, Jain’s fairness index, queue occupancy, and elephant goodput change as buffer size and RTT vary.

4.3 Phase 3: Kernel-Level Improvement

Phase 3 modifies the BBRv1 pacing-gain array in the Linux kernel source and compiles three loadable modules:

- `gain_1.10`: probe-up gain = 1.10×
- `gain_1.15`: probe-up gain = 1.15×
- `gain_1.20`: probe-up gain = 1.20×

We then rerun the full Phase 2 sweep for each gain and compare the results against the stock BBRv1 baseline (1.25×).

4.4 Hypotheses

Our evaluation tests the following hypotheses. The first two are directly motivated by the shallow-buffer and buffer-sensitivity findings reported by Cao et al.[2], while the third extends that prior work into a kernel-level pacing-gain intervention motivated by BBR’s ProbeBW behavior [1, 4]:

1. **Mice flows should experience the worst FCT at the smallest buffer size:** We expect this because a shallow buffer has less room to absorb BBR’s probe-up burst. When the burst overflows the buffer, mice flows arriving mid-probe face packet loss, retransmissions, and long waits before completing. Larger buffers can absorb the burst without this cascading effect [2].
2. **Mice FCT should increase with RTT:** A larger RTT means a larger BDP, which means BBR keeps more data in flight during the probe-up phase. More data in flight builds a larger standing queue. A mouse arriving mid-probe therefore waits longer at higher RTTs than at lower ones. [4].
3. **Reducing BBR’s probe-up gain from 1.25x to a lower value should reduce mice FCT:** The probe-up phase is the direct cause of the queue that mice wait behind. A smaller gain means a smaller overshoot, which should mean a smaller queue, which should mean shorter completion times for mice. We expect the most aggressive reduction, 1.10x, to produce the largest improvement, with diminishing returns as the gain approaches 1.0x. The risk is that reducing the gain too far weakens BBR’s ability to probe for bandwidth, which may hurt elephant throughput as a side effect [1, 4].

5 Experimental Setup

5.1 Experimental Environment

Everything ran on one workstation: WSL2 Ubuntu with kernel 6.6.87.2-microsoft-standard-WSL2, equipped with a 6-core Intel Core i7 processor and 32 GB of RAM. Network emulation used Mininet 2.3.0, which is worth stating explicitly because it relies on real Linux network namespaces and the actual kernel TCP stack rather than a user-space simulation layer. BBR therefore runs as a real kernel congestion-control algorithm. Traffic generation used `iperf3` 3.16, and the experiments were launched from a Python 3.12 virtual environment in the project working directory.

Link bandwidth and delay were controlled via Linux traffic-control primitives: HTB for bandwidth shaping and `netem` for RTT injection. Queue depth was controlled through the bottleneck `qdisc`, and queue occupancy snapshots were collected with `tc -s qdisc`.

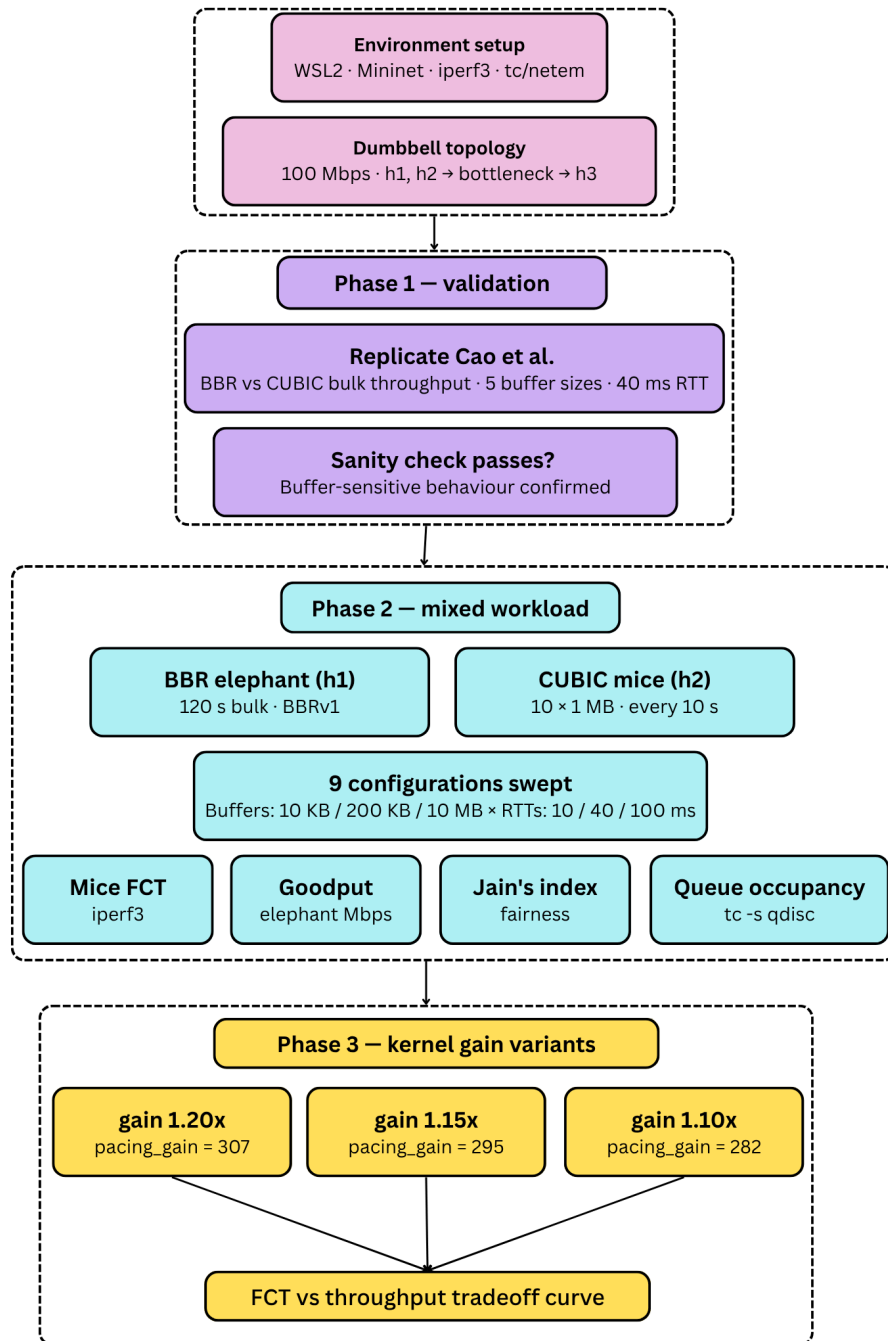


Figure 1: Overview of the project workflow across the three phases, from validation through the gain-variant sweep.

5.2 Network Topology

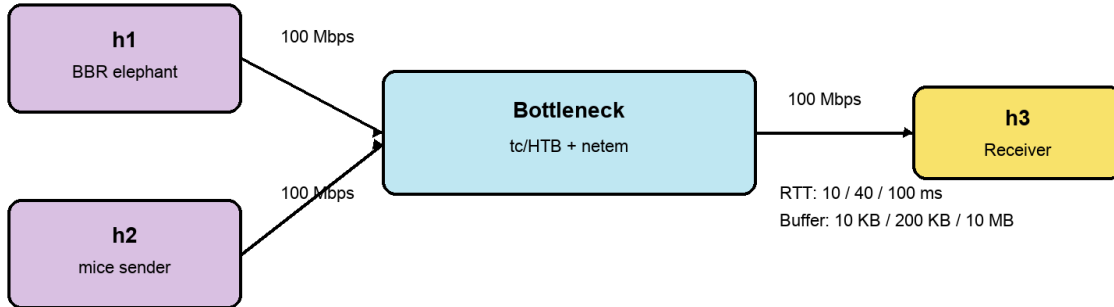


Figure 2: Dumbbell topology used for the project.

We use a dumbbell topology throughout the project: two senders, one shared bottleneck, and one receiver. Host h1 generates the elephant flow, h2 generates the mice flows, and h3 acts as the receiver. The bottleneck link operates at 100 Mbps in all experiments. In the validation phase, the baseline RTT matches the Cao et al. setting of 100 Mbps and 40 ms RTT [2]. In Phases 2 and 3, we sweep RTT across 10, 40, and 100 ms and set the bottleneck buffer to one of three values: 10 KB, 200 KB, or 10 MB.

This topology is intentionally simple. A more complex network could introduce additional path asymmetries or cross-traffic effects, but it would also make it harder to attribute the observed changes specifically to BBR’s probing behavior. The dumbbell setup lets us isolate one question cleanly: what happens to short flows when a single BBR elephant repeatedly builds queue at one shared bottleneck?

The core experiment parameters are summarized in Table 1.

Table 1: Core experimental parameters.

Parameter	Value
Bottleneck bandwidth	100 Mbps
Access-link bandwidth	500 Mbps
RTTs tested	10 ms, 40 ms, 100 ms
Buffer sizes tested	10 KB, 200 KB, 10 MB
Elephant flow	1 BBR flow, 120 s duration
Mice workload	10 sequential flows, 1 MB each
Phase 2 baseline CCA	stock BBRv1
Phase 3 modified CCAs	gains 1.10, 1.15, 1.20

5.3 Phase 1: Baseline Replication

Phase 1 is a validation step. It reproduces the qualitative shallow-buffer throughput trend from Cao et al. at 100 Mbps and 40 ms RTT [2]. We run single-flow bulk transfers of 120 seconds each and collect average throughput from `iperf3` JSON output across five buffer sizes: 10 KB, 50 KB, 200 KB, 1 MB, and 10 MB. The goal is not to claim a full end-to-end reproduction of the IMC 2019 paper, but to confirm that our Mininet and traffic-shaping stack behave sensibly before we move on to mixed workloads.

5.4 Phase 2: Mixed-Workload Characterization

Phase 2 is the core of the empirical study. One BBR elephant flow runs for 120 seconds from h1 to h3. At the same time, h2 sends 10 sequential 1 MB transfers to h3. We sweep three buffer sizes 10 KB, 200 KB, and 10 MB crossed with three RTTs 10 ms, 40 ms, and 100 ms for a total of nine configurations.

For each run, we record four things: mice flow completion time, elephant goodput, Jain’s fairness index, and queue occupancy at the bottleneck. We had also planned to include BBRv3 in the same sweep. That comparison did not happen because the WSL2 kernel used in this project did not expose a usable BBRv3 module, and we could not build one against it cleanly. The placeholder `bbrv3` outputs in the repository document that limitation explicitly.

5.5 Phase 3: Kernel Module Gain Variants

Phase 3 is where the intervention happens. BBR’s probe-up gain is stored inside the `pacing_gain[]` array in the Linux `tcp_bbr.c` source. From that file, we built three custom kernel modules corresponding to alternative probe-up values of approximately 1.10×, 1.15×, and 1.20×. These values are encoded in the kernel source as scaled integers, so the edit is small in code size even though its behavioral effect is measurable.

Each modified version was compiled into its own loadable kernel module `tcp_bbr_gain110.ko`, `tcp_bbr_gain115.ko`, and `tcp_bbr_gain120.ko`. Before each experiment run, the relevant module was loaded with `insmod`, and it was unloaded after that variant completed. We then reran the full nine-configuration Phase 2 sweep for each gain variant, producing 27 additional experiments. The purpose of Phase 3 is therefore not to redesign BBR, but to test whether one narrow kernel-level change can improve mice latency without destroying the throughput behavior that made BBR attractive in the first place.

5.6 Metrics

We use four main metrics:

1. **Mean mice FCT**: average wall-clock completion time across the ten mice flows.
2. **Elephant goodput**: measured from `iperf3` JSON output when available.
3. **Jain’s fairness index**: for throughputs $x_1, \dots, x_n$,

$$J(x_1, \dots, x_n) = \frac{(\sum_i x_i)^2}{n \sum_i x_i^2}.$$

4. **Queue occupancy snapshot**: bottleneck backlog measured through `tc -s qdisc`.

These metrics are complementary rather than redundant. Mice FCT measures user-visible latency directly. Elephant goodput tells us whether the intervention preserves BBR’s main strength. Jain’s fairness index captures how evenly service is shared across the mixed workload. Queue occupancy gives a direct view into the mechanism connecting them. A report focused on only one of these metrics would miss part of the story: low mice FCT with major throughput loss would not be a useful systems result, and good throughput with pathological short-flow delay would miss the central problem this project set out to study.

6 Results and Discussion

6.1 What We Assumed Before Running the Experiments

The project was built around three assumptions about BBR’s operating behavior. These assumptions came directly from the BBR literature and from the mechanics of the ProbeBW cycle, but part of the point of the project was to test how well they held in our own environment rather than simply repeating them.

Table 2: Core assumptions, expected outcomes, and what the experiments actually showed.

Assumption	What we expected to see	What we actually observed
Shallow buffers amplify BBR’s standing-queue problem.	Mice FCT should degrade sharply at 10 KB relative to 200 KB and 10 MB.	Confirmed strongly. The 10 KB regime was catastrophic, while 200 KB and 10 MB were near-ideal by comparison.
Larger RTT should worsen queueing because larger RTprop implies a larger BDP and a larger standing queue.	Mice FCT should rise monotonically with RTT.	Partially confirmed. The RTT trend was visible at 200 KB and 10 MB, but at 10 KB the buffer was so small that all RTTs were already in the failure regime.
Reducing the probe-up gain should meaningfully cut mice FCT while preserving elephant throughput.	The most aggressive reduction, 1.10, was expected to help the most.	Only partially confirmed. All modified gains improved shallow-buffer FCT, and 1.20 produced the largest latency reduction in the canonical data, but missing goodput fields prevent a definitive throughput winner.

Two additional implicit assumptions also turned out to matter:

1. **We assumed that reducing excess pacing gain would translate roughly linearly into FCT reduction.** In reality, once the bottleneck buffer is already too small to absorb any probe-up burst, the experiment enters a buffer-exhaustion regime where loss and timeout behavior dominate. In that regime, lowering the gain still helps, but not nearly as much as a simple queue-size argument would suggest.
2. **We assumed that the best test point for the gain modification would be the smallest buffer.** The experiments showed that 10 KB is the most dramatic failure case, but also the least forgiving one. Because it is so far below the effective in-flight demand of the elephant, even a reduced gain can still overrun the bottleneck. This suggests that the intervention may be even more informative in an intermediate buffer range that we did not sweep.

6.2 Phase 1: Validation, Not Full Reproduction

Table 3 summarizes the Phase 1 throughput measurements. The qualitative trend matches Cao et al.[2]: BBR outperforms CUBIC at very shallow buffers, and both algorithms converge near link rate as the buffer becomes sufficiently large.

Table 3: Phase 1 throughput sanity-check results at 100 Mbps and 40 ms RTT.

Buffer	BBR (Mbps)	CUBIC (Mbps)
10 KB	6.11	2.62
50 KB	92.10	15.38
200 KB	92.93	93.87
1 MB	92.74	93.32
10 MB	92.76	92.35

This phase should still be interpreted conservatively. Our validation note explicitly treats it as a sanity check rather than a faithful end-to-end reproduction of Cao et al.’s IMC 2019 study [2]. The original paper uses a wider parameter sweep, repeated trials, and a different shaping methodology [2]. Still, Phase 1 established that our experiment stack produces nontrivial, buffer-sensitive congestion behavior.

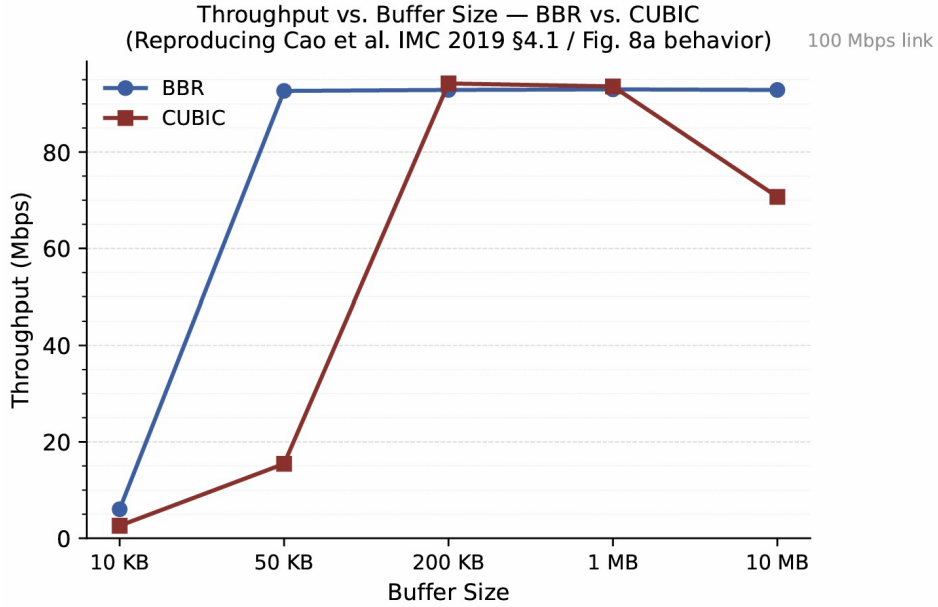


Figure 3: Throughput versus buffer size for BBR and CUBIC at 100 Mbps and 40 ms RTT, reproducing the qualitative trend reported by Cao et al.

6.3 Phase 2: Mixed-Workload Characterization

Phase 2 isolates the operational regime that motivates the rest of the project. Our main assumption here was that a single long-lived BBR elephant would be enough to expose the latency cost of BBR’s standing queue when short flows shared the same bottleneck. Figure 4 shows that this assumption was correct, and in fact the effect was stronger than we initially expected. The dominant pattern is a sharp degradation at the 10 KB buffer.

Phase 2: Mice FCT Heatmap (BBR Elephant + Mice)

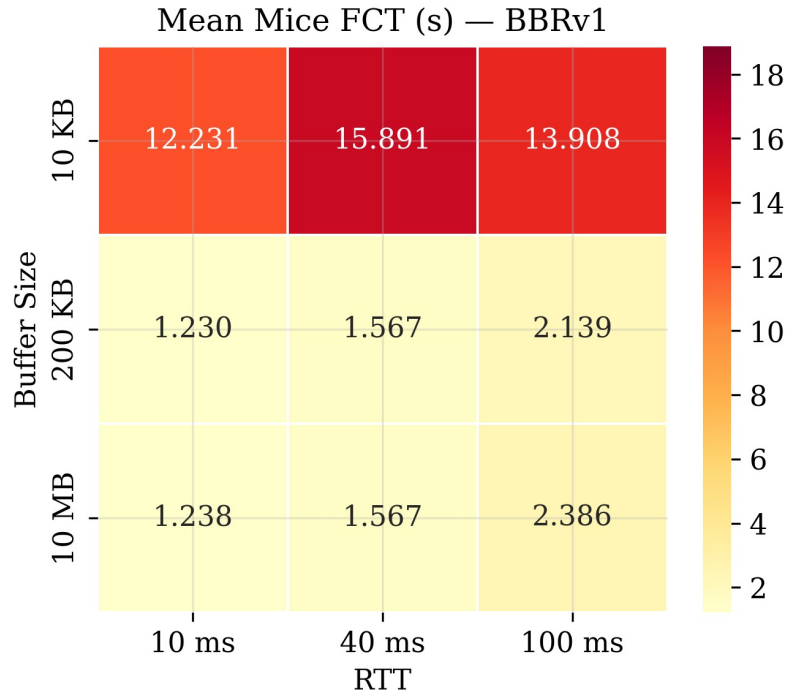


Figure 4: Phase 2 mice-FCT heatmap for the mixed-workload baseline.

The mean FCT values for stock BBRv1 are:

- **10 KB buffer:** 12.23 s at 10 ms RTT, 15.89 s at 40 ms RTT, 13.91 s at 100 ms RTT.
- **200 KB buffer:** 1.23 s at 10 ms RTT, 1.57 s at 40 ms RTT, 2.14 s at 100 ms RTT.
- **10 MB buffer:** 1.24 s at 10 ms RTT, 1.57 s at 40 ms RTT, 2.39 s at 100 ms RTT.

This is the key empirical cliff in the project: moving from 10 KB to 200 KB reduces mice FCT by roughly an order of magnitude in the lowest-RTT setting. In other words, the mice-flow problem is not a mild degradation but a regime change. Once the buffer is large enough to stop chronic overrun, mice FCT collapses back toward a much more reasonable operating range.

Fairness follows the same pattern. At 10 KB, Jain's fairness index is only 0.267 at 10 ms RTT, while at 200 KB and 10 MB it is effectively 1.0 in the low-RTT regime. This low fairness is itself informative: it implies that not all mice are hurt equally. Some mice happen to arrive in a relatively favorable part of the elephant's pacing cycle and complete faster, while others arrive during or immediately after probe-up and pay a much larger queueing penalty. The result is both high latency and high variability, which is exactly the kind of behavior that makes user-facing systems unstable.

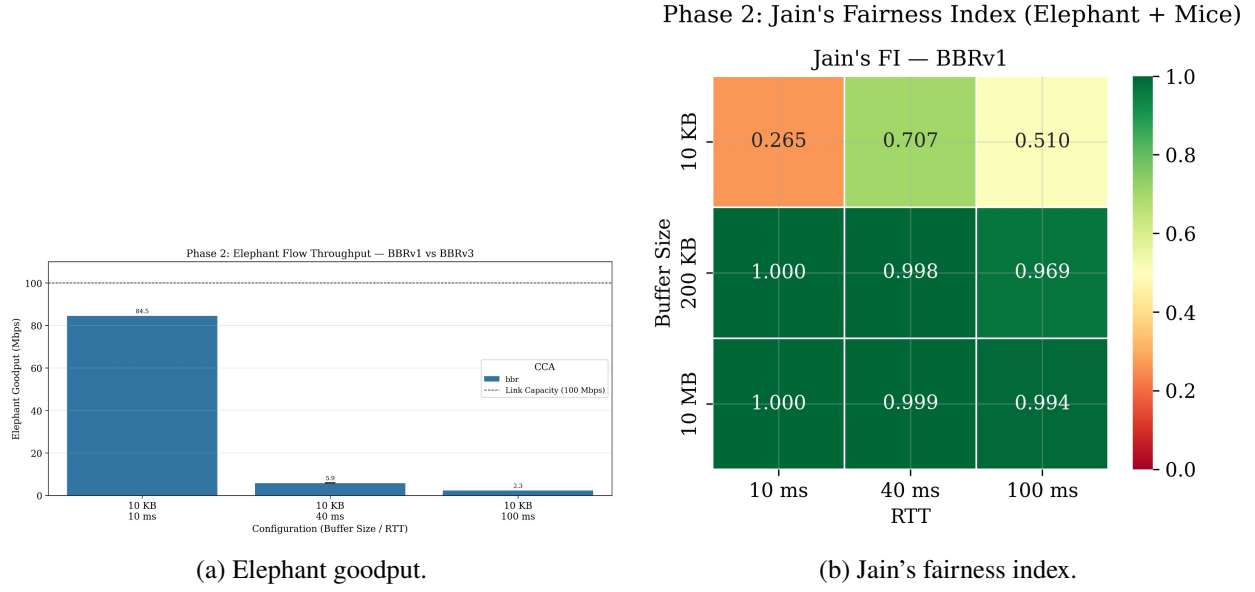


Figure 5: Phase 2 summary plots.

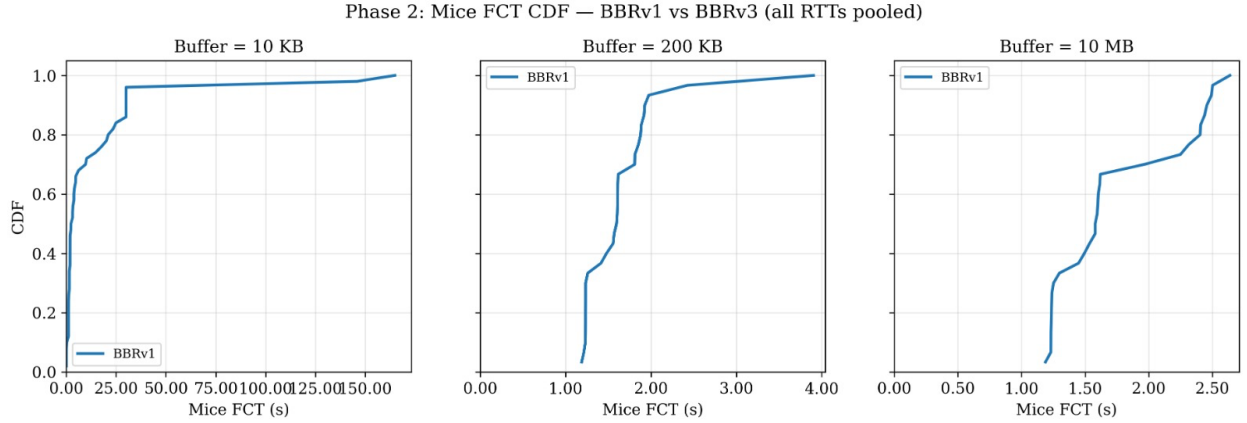


Figure 6: Cumulative distribution of mice FCT across all RTTs pooled, for each buffer size.

Elephant goodput at 10 KB also varies strongly with RTT: 84.53 Mbps at 10 ms, 5.89 Mbps at 40 ms, and 2.32 Mbps at 100 ms. Together, these results support the central diagnosis of the project: when the buffer is extremely shallow, BBR's probe behavior creates a regime where the elephant remains active while mice flows experience severe delay and poor fairness.

The RTT trend deserves separate interpretation. We expected higher RTT to worsen the queueing effect because a larger RTTprop means a larger BDP and therefore a larger amount of data in flight during probing. The data partially supports that expectation, but only outside the most extreme shallow-buffer case. At 200 KB and 10 MB, FCT increases from the 10 ms setting to the 100 ms setting, which is consistent with a larger standing-queue burden. At 10 KB, however, the buffer is already so constrained that the experiment is effectively dominated by buffer exhaustion rather than by steady-state queue scaling. In that regime, all three RTT points are bad enough that the finer-grained RTT effect becomes less visible.

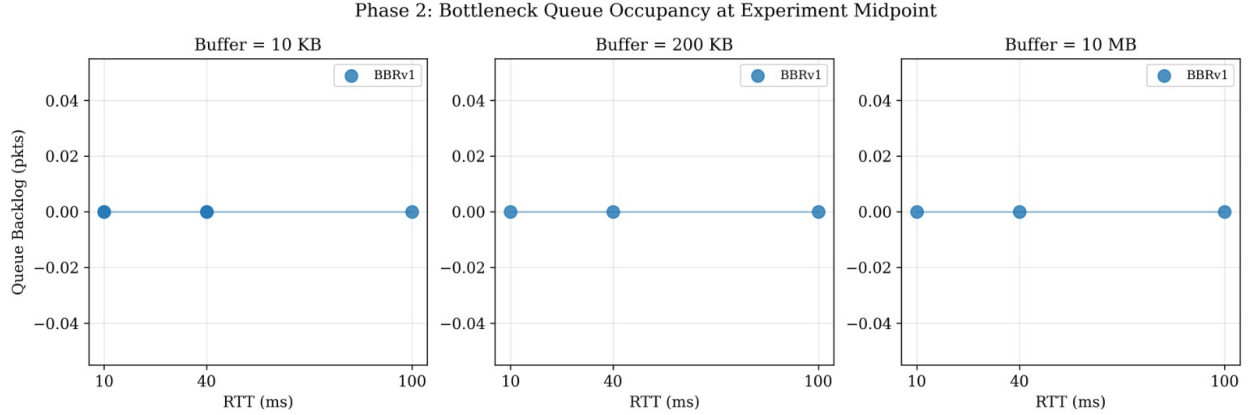


Figure 7: Bottleneck queue occupancy at the experiment midpoint. This single steady-state snapshot is less informative than the FCT plots because it does not capture the transient queue bursts that dominate short-flow latency.

The main Phase 2 conclusion is therefore stronger than “BBR hurts mice.” What we actually observed is more specific:

1. the penalty is concentrated in the shallow-buffer regime,
2. the penalty affects both average latency and fairness,
3. and once the buffer is no longer the binding constraint, the same mixed workload becomes much more benign.

We attempted to include BBRv3 in Phase 2, but the WSL2 kernel used in this project did not expose a usable BBRv3 module. As a result, our validated Phase 2 analysis focuses on BBRv1 only. This is a platform limitation rather than a design choice.

6.4 Phase 3: Pacing-Gain Modification

Phase 3 evaluates whether a small reduction in BBR’s probe-up gain can improve the Phase 2 baseline. Our starting assumption was straightforward: if the queue is created by the $1.25\times$ probe-up step, then reducing that overshoot should reduce the queue, which should in turn reduce mice FCT. Figure 8 shows the main tradeoff curve, and Figure 9 breaks down mean FCT by gain.

The averaged mice-FCT results across RTTs are shown in Table 4. The most important observation is that every modified gain improves shallow-buffer mice FCT relative to the stock $1.25\times$ baseline. The strongest latency reduction comes from $1.20\times$, but the throughput side of the story must be interpreted more cautiously because several runs are missing elephant-goodput values.

Table 4: Phase 3 mean mice FCT (seconds), averaged over RTTs.

Pacing gain	10 KB	200 KB	10 MB
1.10	12.3595	1.5439	1.6629
1.15	12.3149	1.5935	1.6768
1.20	12.0789	1.6212	1.7130
1.25 (baseline)	14.0302	1.6454	1.7302

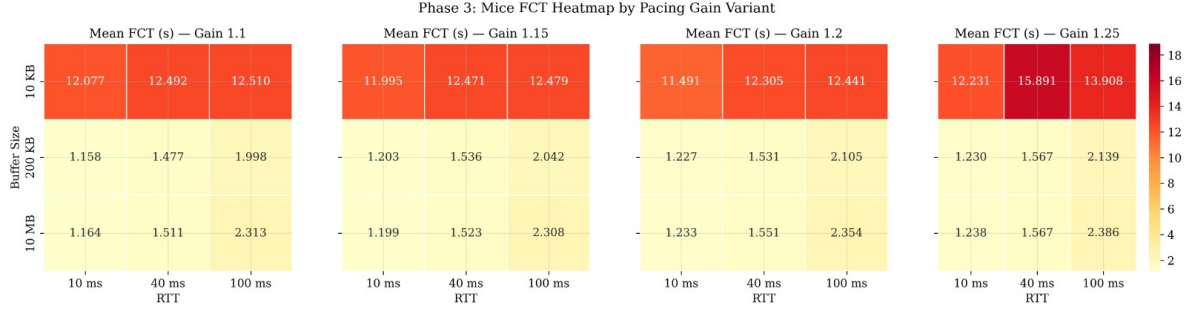


Figure 8: Mean mice FCT versus elephant goodput at the 10 KB buffer, with each point representing one RTT configuration. Red points are the BBRv1 baseline, and green shades are the three modified gain variants.

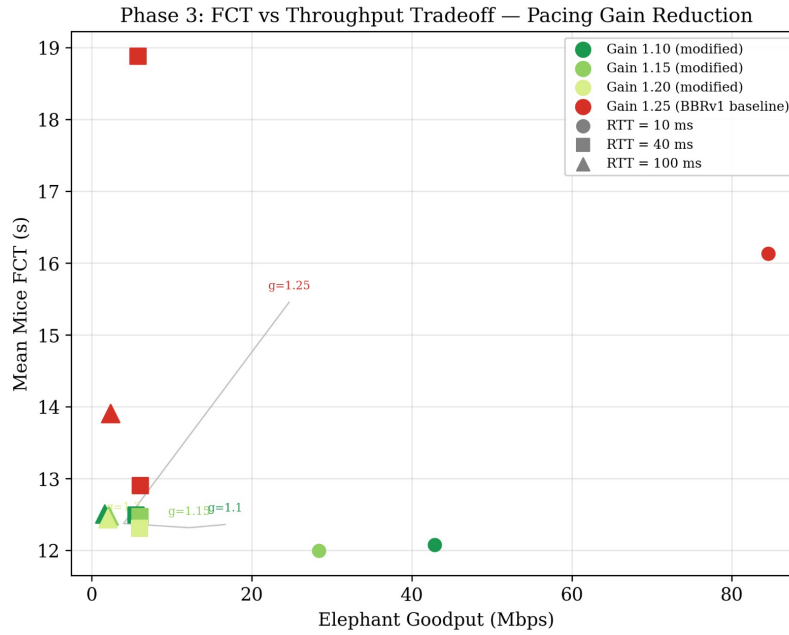


Figure 9: Mean mice FCT averaged across all three RTTs, plotted against pacing gain, for each buffer size.

Relative to the baseline, the FCT improvements are:

- **Gain 1.10:** +11.9% at 10 KB, +6.2% at 200 KB, +3.9% at 10 MB.
- **Gain 1.15:** +12.2% at 10 KB, +3.2% at 200 KB, +3.1% at 10 MB.
- **Gain 1.20:** +13.9% at 10 KB, +1.5% at 200 KB, +1.0% at 10 MB.

These results confirm the basic direction of the hypothesis but not its strongest form. Reducing the probe-up gain does help, but the relationship is not monotonic, and the largest reduction is not automatically the best one. The strongest mice-FCT improvement at the shallow 10 KB buffer comes from 1.20, not 1.10. At the larger buffers, the differences shrink substantially, which indicates that the gain modification matters most when queue pressure is already the dominant problem.

There are two plausible reasons for this. First, the 10 KB case is so extreme that it is not just a standing-queue experiment; it is also a buffer-exhaustion experiment. Once the bottleneck is already being overrun,

lowering the gain cannot fully eliminate the failure mode. Second, reducing the gain changes not only how much queue is created but also how effectively BBR probes for bandwidth. That makes the intervention a tuning problem rather than a one-directional improvement knob: a lower gain can reduce queue pressure while also changing how the elephant tracks capacity.

The elephant-goodput side of Phase 3 is informative but incomplete. Available 10 KB measurements include:

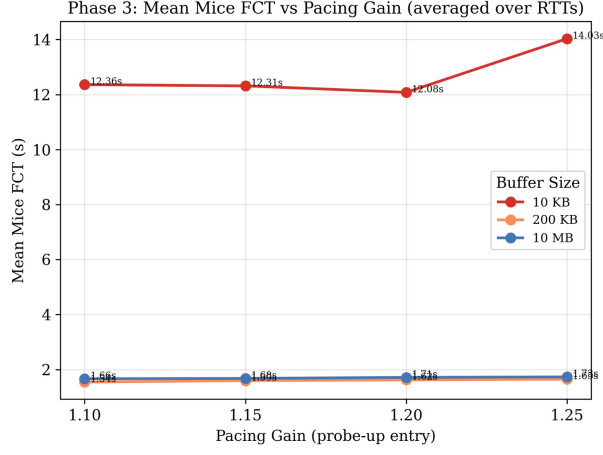
- **1.10:** 15.72 Mbps
- **1.15:** 12.09 Mbps
- **1.20:** 3.97 Mbps in the averaged plot, but this average is based only on the 40 ms and 100 ms runs because the 10 ms goodput field is missing.
- **1.25 baseline:** 24.66 Mbps

This means the goodput figure should be read qualitatively, not as a fully apples-to-apples ranking across gains. The canonical dataset supports a strong latency conclusion: all modified gains help at 10 KB, and 1.20 helps the most on average. It supports a weaker throughput conclusion: available goodput measurements remain noisy and incomplete because several runs saved valid FCT samples but left `elephant_goodput_mbps = None`. We therefore avoid claiming a definitive throughput-optimal gain variant from this dataset alone.

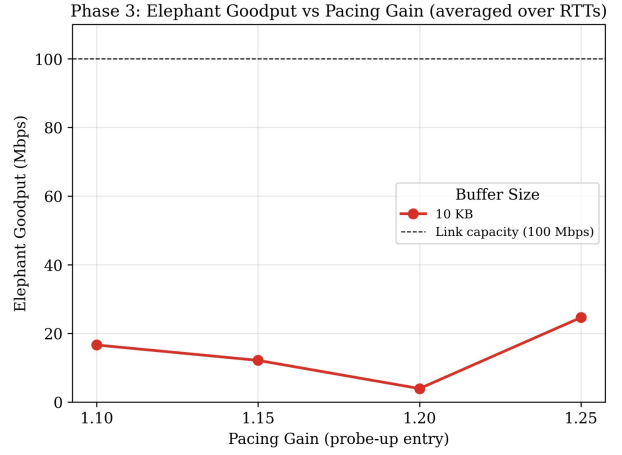
Another important observation is where the gain modification does *not* matter very much. At 200 KB and 10 MB, all four gain settings are much closer together than at 10 KB. This tells us that the intervention is specifically useful when queue pressure is already a bottleneck concern. When the buffer is large enough to absorb the workload comfortably, the stock algorithm is already near its local optimum and changing the gain yields limited benefit. Figure 11b makes this visible directly: the largest changes appear in the 10 KB row, while the larger buffers remain tightly clustered across all four gain settings.

Taken together, the Phase 3 results refine our original hypothesis into a more defensible claim:

Reducing BBRv1's probe-up gain can improve mixed-workload mice latency, but the best setting is not simply the most aggressive one, and the benefit is largest in queue-sensitive regimes. In the canonical dataset, the clearest result is that 1.20 gives the largest shallow-buffer FCT reduction; the throughput ranking is less certain because of missing goodput fields.

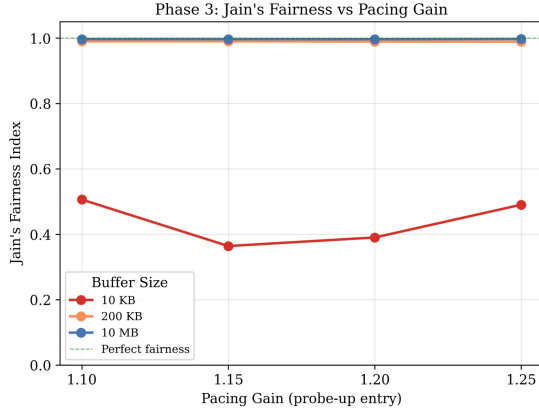


(a) Elephant goodput averaged across all three RTTs, plotted against pacing gain for the 10 KB buffer where goodput data was available.

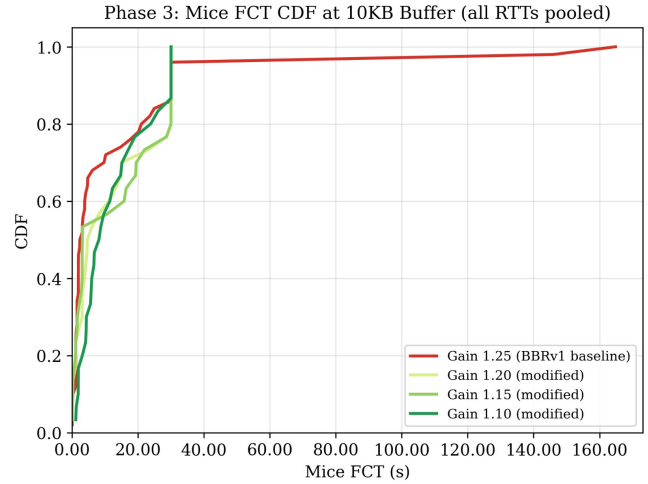


(b) Jain's fairness index across the ten mice FCT values, plotted against pacing gain for each buffer size.

Figure 10: Additional averaged Phase 3 views across pacing gain.



(a) Cumulative distribution of mice FCT at the 10 KB buffer across all RTTs pooled, comparing the BBRv1 baseline against the three modified gain variants.



(b) Mean mice FCT (seconds) across all nine buffer-RTT configurations for each of the four pacing gain values tested: 1.10x, 1.15x, 1.20x, and the 1.25x baseline.

Figure 11: Additional Phase 3 latency views.

6.5 Overall Interpretation

Across all three phases, the data supports a coherent story.

Phase 1 showed that our stack reproduces the expected shallow-buffer sensitivity qualitatively. Phase 2 then showed that this sensitivity becomes operationally severe once a long-lived BBR elephant shares the bottleneck with short flows. Phase 3 finally showed that the behavior is tunable at the kernel level, but also that the tuning space is subtler than a simple “lower is better” rule.

The most important lesson is that two different regimes are present in the data:

1. **Standing-queue regime:** when the buffer is large enough for ProbeBW to leave a persistent queue without immediately collapsing into chronic loss, RTT and pacing gain both matter in a smoother way.
2. **Buffer-exhaustion regime:** when the buffer is too shallow, mice latency is dominated by an almost pathological interaction between the elephant's overshoot and the lack of queue space. In this regime, parameter tuning still helps, but only within a hard floor imposed by the tiny buffer.

That distinction explains why our strongest hypothesis was only partially confirmed. The idea behind the gain reduction was correct, but our smallest buffer point was harsh enough that it limited how much improvement any modified gain could deliver. The project therefore ends with both a conclusion and a next step: the intervention works, but it should be evaluated more densely in the intermediate buffer region where queue formation is significant but not yet fully catastrophic.

7 Limitations and Threats to Validity

This project has four main limitations.

1. **Platform limitation.** BBRv3 was unavailable on our WSL2 kernel, so we could not perform a clean BBRv1-versus-BBRv3 comparison.
2. **Phase 1 scope.** The validation sweep is narrower than Cao et al.'s original study and should be treated as a sanity check rather than a full reproduction.
3. **Measurement noise.** Some iperf3 runs produced missing elephant goodput due to a known race in JSON output generation. Our analysis scripts load valid FCT data even when those goodput fields are missing.
4. **External validity.** Our Mininet dumbbell is deliberately simple. It isolates the elephant-mice interaction cleanly, but it does not model large multi-bottleneck WANs or production datacenter fabrics.

These limitations do not invalidate the core finding, but they do constrain its scope: our conclusions are strongest for the single-bottleneck mixed-workload setting we studied.

8 Conclusion

This project set out to answer a specific question: can a small reduction in BBRv1's probe-up pacing gain meaningfully improve short-flow latency in a mixed elephant-mice workload? The honest answer is no, and understanding why is the most useful thing this study produced.

We built the project around three hypotheses:

- **Hypothesis 1: Shallow buffers would be the worst case for mice FCT.** This was confirmed, and more strongly than we expected. But then again, any congestion control algorithm would fail with a buffer size as small as 10 KB. At 10 KB, a 1 MB transfer that should finish in under 100 ms instead took 12 to 19 seconds. Fairness collapsed to a Jain index of 0.265. Moving to 200 KB dropped FCT to around 1.2 to 2.4 seconds and restored fairness to near 1.0. The cliff between these two regimes is the central empirical finding of the project.
- **Hypothesis 2: Mice FCT would increase with RTT as larger RTprop produces a larger standing queue.** This was partially confirmed. At 200 KB and 10 MB, FCT did rise monotonically from 10 ms to 100 ms RTT, consistent with the standing-queue prediction. At 10 KB, the effect was invisible because buffer exhaustion dominated entirely. All three RTT values produced catastrophic results regardless.

- **Hypothesis 3: Reducing the probe-up gain would improve mice FCT.** This was not confirmed in any practically useful sense. We genuinely expected that lowering the gain from 1.25x to 1.10x, 1.15x, or 1.20x would reduce the probe-up burst and proportionally reduce the queue mice had to wait behind. It did not work out that way. At 10 KB, the best variant shaved roughly 2 seconds off a completion time that was already 12 to 16 seconds. At 200 KB and 10 MB, differences across all gain values were under half a second. None of these numbers represent a meaningful improvement for a transfer that should take under 100 milliseconds in the first place.

Table 5: Summary of hypotheses and final outcomes.

Hypothesis	Final outcome
Shallow buffers would be the worst case for mice FCT.	Confirmed strongly. The 10 KB buffer produced catastrophic mice latency and poor fairness, while 200 KB and 10 MB were much more stable.
Mice FCT would increase with RTT because larger RTprop produces a larger standing queue.	Partially confirmed. The RTT trend is clear at 200 KB and 10 MB, but at 10 KB it is masked by extreme shallow-buffer behavior.
Reducing the probe-up gain would meaningfully improve mice FCT.	Only weakly confirmed. All modified gains helped somewhat, but the improvement was modest, and 1.20 gave the best average shallow-buffer result in this dataset.

The reason the intervention failed is structural. At 10 KB, the buffer is so far below the elephant’s BDP that the failure mode is buffer exhaustion, not standing-queue buildup. Once the buffer is in exhaustion territory, the probe-up gain value becomes largely irrelevant: even the most conservative gain still overwhelms a 10 KB buffer completely. We were targeting the right mechanism but the wrong regime. The gain reduction is a standing-queue intervention, and our smallest buffer was never in the standing-queue regime to begin with.

What this experiment tells us, in the end, is less about BBR’s gain parameter and more about where the interesting problem actually lives. The 50 KB to 150 KB buffer range is what we should have swept. At 10 KB, the results are not BBR-specific: any congestion control algorithm would hurt short flows at a buffer that small. At 200 KB, the problem disappears on its own. Somewhere in between, the standing-queue effect transitions from being masked by exhaustion to being the dominant factor, and that is precisely where a probe-up gain reduction might produce a tradeoff worth measuring. We know from our data that 50 KB is likely still too small and that the meaningful threshold is somewhere closer to 100 KB, but pinning that down precisely is work we did not do.

The experiment did not produce the result we hoped for. But it produced something arguably more useful: a clear picture of why the hypothesis failed, where the gap in our parameter sweep is, and what a better-targeted follow-up study would look like.

References

- [1] N. Cardwell, Y. Cheng, C. S. Gunn, S. H. Yeganeh, and V. Jacobson. BBR: Congestion-based congestion control. *Communications of the ACM*, 60(2), 2017.
- [2] Y. Cao, M. Jain, K. Sharma, A. Bhattacharya, and P. Singla. When to use and when not to use BBR: An empirical analysis and evaluation study. In *Proceedings of ACM IMC*, 2019.
- [3] M. Ware, J. Vašíček, R. Ionescu, H. Bhullar, and C. Godsalve. Beyond Jain’s fairness index: Setting the bar for the assessment of congestion control algorithms. In *Proceedings of ACM IMC*, 2019.
- [4] S. Scherrer, M. Legner, A. Singla, and S. Schmid. Model-based insights on the performance, fairness, and stability of BBR. In *Proceedings of ACM IMC*, 2022.
- [5] T. Song, J. Qian, H. Cui, Z. Fei, and H. Jiang. BBR-GC: Enabling fairness via gain control for BBR. *IEEE Access*, 2021.